Problem Chosen	Team Control Number
A	**758**

Controlled Descent and Propellant Estimation for a First-Stage Rocket Booster

Summary

We model a first-stage booster on a nonrotating spherical Earth with coupled translation, pitch, aerodynamics, and propellant consumption. For a 130 t dry mass and a 15° climb angle at separation, the lowest verified inventory is 25.97 t, comprising 5.77 t of methane and 20.20 t of liquid oxygen. The booster coasts for 181 s and ignites once at 6.2 km altitude, burning 20.89 t and retaining the required 5 t reserve. It touches down upright after 213.25 s, with the ship at 284.12 km downrange, a downward speed of 1.51 m/s, and a horizontal speed of 0.003 m/s. Independent replay meets the touchdown limits at steps down to 0.00125 s. Among the parameters varied without a pressure cap, the separation direction has the largest effect. Level and 30° separations use 24.32 and 32.46 t, respectively. Varying effective specific impulse from 300 to 360 s gives 28.48 to 23.98 t, while the tested crossflow coefficients change the inventory by less than 0.5 t. The nominal descent reaches 293 kPa dynamic pressure and an estimated 130–250 kW/m^2 stagnation heat flux before ignition. With an extra braking burn, the lowest verified inventories for 200 and 150 kPa caps are 37.30 and 50.17 t. These capped values are feasible upper estimates. All results are conditional on the reference rigid-body, stationary-deck, and ideal-actuator assumptions.

Contents

1 Introduction **3**
1.1 Background . . . 3
1.2 Restatement of the Problem . . . 3

2 Assumptions **3**

3 Notation **5**

4 Model Force Analysis **5**
4.1 Gravity and Standard Atmosphere . . . 5
4.2 Axial and Crossflow Aerodynamics . . . 6
4.3 Reference Propulsion Parameters . . . 6

5 Model Motion Analysis **7**
5.1 Momentum Balance and Pitch Dynamics . . . 7
5.2 Spherical-Earth Correction . . . 8
5.3 Initial Conditions and Contact Geometry . . . 8
5.4 Analytic Baselines and Scale Estimates . . . 9
5.5 Numerical Integration and Local Powered-Braking Check . . . 9

6 Landing Strategy and Propellant Estimation **10**
6.1 Spherical-Earth Reference Solution . . . 10
6.2 Search and Minimum-Propellant Formulation . . . 12
6.3 Planar Comparison and Inventory Accounting . . . 12
6.4 Dependence on the Separation Direction . . . 12

7 Sensitivity and Flight Loads **13**
7.1 Dependence of the Propellant Budget on Assumptions . . . 13
7.2 Loads Along the Nominal Descent . . . 14
7.3 Aerodynamic Heating Estimate . . . 15
7.4 Cost of Limiting Dynamic Pressure . . . 15
7.5 A Limited Guidance Extension . . . 16
7.6 Summary of the Parameter Studies . . . 16

8 Model Evaluation **16**
8.1 Internal Verification and Numerical Convergence . . . 16
8.2 Strengths and Weaknesses . . . 17

9 Discussion and Conclusions **18**
9.1 Attitude Stability and Vertical Touchdown . . . 18
9.2 Loads and Scope . . . 18
9.3 Earth Curvature and Rotation . . . 18
9.4 Ship Placement, Wind, and Deck Motion . . . 18
9.5 Conclusions and Practical Recommendations . . . 18

A Reproducibility and Source Code **20**
A.1 Core Source Listings . . . 21

Report on the Use of AI **31**

1 Introduction

1.1 Background

A reusable first stage has to come back intact. After separation it is still moving at several times the speed of sound, high in the atmosphere, and it must lose that speed, turn upright, and settle onto a landing site gently enough to be flown again. For a sea landing the recovery ship can be placed under the descent path, so the booster does not have to fly back toward the launch site. What remains is to brake, to keep the attitude under control, and to touch down slowly with some propellant in reserve. The propellant carried for this descent is mass that the first stage cannot use during ascent, so the size of the descent budget matters for the whole vehicle.

This paper estimates that budget. Sections 2–5 set out the assumptions and the force and motion model, including the correction for a spherical Earth. Section 6 finds a minimum-propellant landing strategy, reports the separation inventory, and shows how it depends on the separation direction. Section 7 examines engine efficiency, aerodynamic drag, and the loads along the descent, and the final sections evaluate the model and state the conclusions.

1.2 Restatement of the Problem

At separation the booster is at 70 km altitude and moves at 6000 km/h. It is 65 m tall and 7.0 m in diameter, and the problem figure gives its dry mass as 130000 kg. It burns liquid oxygen and methane. The recovery ship may be placed wherever is convenient. We are asked to describe a safe descent and a controlled vertical landing and to estimate how much fuel the booster must carry at separation [10]. Its total mass at separation is therefore the dry mass plus the propellant on board.

The problem gives the speed at separation but not its direction. The direction sets how long the booster stays aloft, how far it travels downrange, and its state when the landing burn begins. It is fixed by the ascent mission, not by the recovery plan, so we treat it as an input, with 15° above horizontal as the reference (Section 6.4). Throughout, *propellant* means the combined mass of liquid oxygen and methane, *fuel* means the methane alone, and the *inventory* is the propellant mass on board at separation.

2 Assumptions

- **Vehicle.** The booster is a rigid circular cylinder, 65 m long and 7 m in diameter. Its 130 t dry mass is distributed uniformly for the pitch inertia, and the remaining propellant is treated as a point mass at the center of mass (Section 5). Pitch rotation is modeled because the offset between the center of mass and the center of pressure creates aerodynamic moments, so an upright touchdown has to follow from the dynamics. Structural and thermal capacity are outside the study; the loads are reported as requirements.

- **Earth and atmosphere.** The reference calculation uses a nonrotating spherical Earth with inverse-square gravity and the 1976 U.S. Standard Atmosphere, continued isothermally above 84.852 km geopotential altitude where needed. A flat-Earth

version of the same equations provides starting guesses and the earlier planar comparisons. Motion stays in the vertical plane of the separation velocity, and the nominal case has no wind.

- **Separation state.** The booster separates at 70 km altitude and 6000 km/h. The flight-path angle, which the problem does not give, is 15° above horizontal in the reference case and 0° or 30° in the alternatives. The body axis is initially aligned with the velocity and the pitch rate is zero.

- **Propulsion.** One engine provides up to 2.846 MN, throttleable between 40% and 100%, with a ±10° gimbal, an effective specific impulse of 330 s, and an oxygen-to-methane mass ratio of 3.5, so methane is 1/4.5 of the propellant mass. At most three ignitions are allowed. Throttle and gimbal respond instantaneously, with no ignition delay or minimum burn time. There are no fins or reaction-control thrusters, so the attitude evolves passively while the engine is off.

- **Aerodynamics.** Axial and crossflow forces use constant coefficients $C_a = 0.8$ and $C_n = 1.2$, with the center of pressure $0.05L$ ahead of the center of mass.

- **Ship and touchdown.** The ship is stationary, with a level deck at sea level, and may be placed anywhere along the ground track; each scenario gets its own position. The booster has no landing gear. A landing is accepted when, at first contact, the booster is within 5 m of the ship's position, moves less than 1 m/s horizontally and 2 m/s vertically, tilts less than 2° from upright, rotates slower than 1°/s, and retains at least 5 t of usable propellant. Unusable residual propellant is taken as zero.

- **Parameter studies.** We vary the separation angle (0°, 15°, 30°), the effective specific impulse (300, 330, 360 s), and the crossflow coefficient (0.8, 1.0, 1.2, 1.6), one at a time. The axial coefficient, pressure-center offset, thrust, and reserve are not varied.

3 Notation

Symbol	Meaning	Unit
x	Downrange distance in the flat-Earth model	m
s, h	Surface arc distance and radial COM altitude	m
r, R_E	Distance from Earth center, Earth radius	m
v_x, v_h	Local horizontal and radial velocity	m/s
θ, ω	Pitch from local horizontal, local pitch rate	rad, rad/s
$m, m_d, m_{p,0}$	Total, dry, and separation propellant mass	kg
T, u_T, δ	Thrust, throttle fraction, gimbal angle	N, 1, rad
I_{sp}, g_0	Effective specific impulse, standard gravity	s, m/s^2
L, D, R_b	Body length, diameter, radius	m
A_{end}	End (axial reference) area	m^2
C_a, C_n	Axial and crossflow force coefficients	1
Δl	Pressure-center offset from COM	m
ξ	Axial distance of an aerodynamic strip from the COM	m
I_d	Dry pitch moment of inertia	kg m^2
ρ	Air density	kg/m^3
γ_0	Initial flight-path angle	rad
q	Dynamic pressure, $\rho U^2/2$	Pa

4 Model Force Analysis

4.1 Gravity and Standard Atmosphere

We first write the model in a flat downrange–vertical plane; Section 5.2 then adds Earth curvature for the reference calculation. Position refers to the center of mass (COM), and the pitch angle θ is the angle of the body axis, from tail to nose, above the horizontal. Gravity acts at the COM with magnitude

$$g(h) = g_0 \left(\frac{R_E}{R_E + h} \right)^2, \qquad g_0 = 9.80665 \text{ m/s}^2, \quad R_E = 6356766 \text{ m}. \tag{1}$$

At 70 km this gives $g = 9.5942$ m/s^2, about 2.17% below g_0.

Atmospheric properties follow the lower layers of the 1976 standard atmosphere [3]. Geometric altitude h is converted to geopotential altitude $H = R_E h/(R_E + h)$. In a layer with base (H_b, T_b, p_b) and temperature gradient λ,

$$T_a = T_b + \lambda(H - H_b), \tag{2}$$

$$p_a = \begin{cases} p_b (T_a/T_b)^{-g_0/(R_a \lambda)}, & \lambda \neq 0, \\ p_b \exp[-g_0(H - H_b)/(R_a T_b)], & \lambda = 0, \end{cases} \tag{3}$$

$$\rho = \frac{p_a}{R_a T_a}, \qquad R_a = 287.05287 \text{ J/(kg K)}. \tag{4}$$

These expressions follow from hydrostatic balance, $dp_a/dH = -\rho g_0$, and the ideal-gas law. The tabulated layers end at $H = 84.852$ km. Above that altitude we continue the last layer isothermally, which matters only for the steep separation case of Section 6.4.

4.2 Axial and Crossflow Aerodynamics

The cylindrical approximation gives $A_{end} = \pi D^2/4 = 38.4845$ m^2 and side area $DL = 455$ m^2. Define body directions

$$\mathbf{e} = (\cos\theta, \sin\theta), \quad \mathbf{n} = (-\sin\theta, \cos\theta), \quad \mathbf{u} = \mathbf{v} - \mathbf{w}, \quad u_a = \mathbf{u}\cdot\mathbf{e}, \quad u_n = \mathbf{u}\cdot\mathbf{n}. \tag{5}$$

We split the aerodynamic force into axial and crossflow parts instead of applying one coefficient to a changing projected area, because flow across a cylinder behaves differently from flow along it [2]. The constant coefficients are engineering estimates and are not fitted to that report.

The axial force is

$$F_a = -\tfrac{1}{2}\rho C_a A_{end} u_a |u_a|. \tag{6}$$

For an equivalent crossflow strip at signed axial distance ξ from the COM, rotation adds the local normal velocity $\omega\xi$. Thus

$$dF_n = -\tfrac{1}{2}\rho C_n D(u_n + \omega\xi)|u_n + \omega\xi|\, d\xi. \tag{7}$$

With N midpoint strips of length $\Delta\xi = L/N$ and $\xi_i = -L/2 + (i - 1/2)\Delta\xi + \Delta l$,

$$\mathbf{F}_D = F_a\mathbf{e} + \sum_i F_{n,i}\mathbf{n}, \qquad M_a = \sum_i \xi_i F_{n,i}. \tag{8}$$

Shifting the strips by Δl represents the uncertain position of the center of pressure. Density is evaluated at the COM, and its variation along the 65 m body is neglected. The axial force acts along the centerline. Figure 1 defines the forces and signs.

Without rotation, $M_a = \Delta l F_n$, so even a uniform wind produces a moment when $\Delta l \neq 0$. For a centered distribution in pure rotation, direct integration gives

$$M_a = -\frac{\rho C_n D L^4}{64}\omega|\omega|. \tag{9}$$

which always opposes the rotation. More generally, the power delivered by the air-relative forces satisfies

$$\mathbf{F}_D\cdot\mathbf{u} + M_a\omega = -\tfrac{1}{2}\rho C_a A_{end}|u_a|^3 - \tfrac{1}{2}\rho C_n D\sum_i \Delta\xi\,|u_n + \omega\xi_i|^3 \le 0. \tag{10}$$

The aerodynamic forces therefore always remove energy from the motion relative to the air, which gives a useful check on signs and implementation. In a wind, the energy measured in the ground frame can still increase.

4.3 Reference Propulsion Parameters

We set the maximum thrust to 2.846 MN, the published thrust of one BE-4 methane–oxygen engine [1], and treat it as a representative single landing engine; the problem does not specify the engine. The effective specific impulse of 330 s, the 40% minimum throttle, and the 10° gimbal limit are design assumptions. The oxygen-to-methane mass ratio of 3.5 follows a methane–oxygen mission study [4]; we do not use that study's vacuum specific impulse, because the landing burn takes place low in the atmosphere. The aerodynamic reference values are $C_a = 0.8$, $C_n = 1.2$, and $\Delta l = 0.05L$.

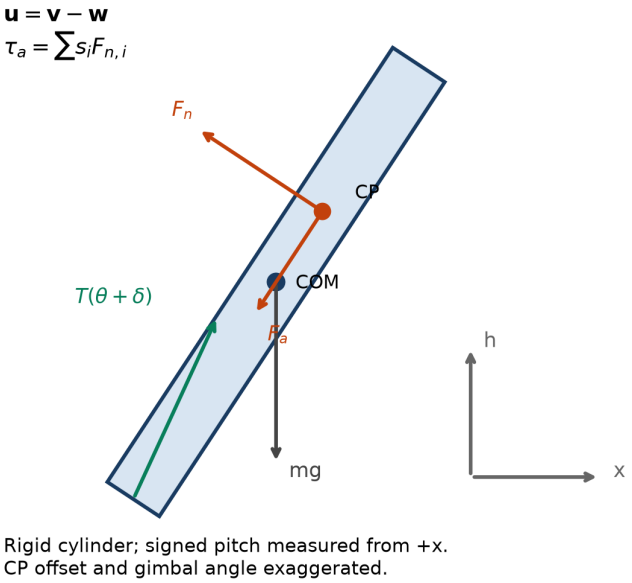


Figure 1: Geometry and force conventions. The pressure-center offset is exaggerated. A positive gimbal angle at the tail produces a negative pitch moment.

The standard effective-exhaust-velocity definition gives $c = I_{sp} g_0$ and

$$\dot{m} = -T/c, \qquad T = 0 \quad \text{or} \quad 0.4T_{max} \le T \le T_{max}. \tag{11}$$

The flow consumes oxygen and methane in the fixed mixture ratio. The variation of thrust with ambient pressure and the engine transients are absorbed into the effective parameters.

5 Model Motion Analysis

5.1 Momentum Balance and Pitch Dynamics

The state is $\mathbf{y} = (x, h, v_x, v_h, m, \theta, \omega)$. Because the thrust already accounts for the momentum carried away by the exhaust, the translational balance is $m\dot{\mathbf{v}} = m\mathbf{g} + \mathbf{F}_D + \mathbf{F}_T$, with no separate $\dot{m}\mathbf{v}$ term. In the flat-Earth model the equations are

$$\dot{x} = v_x, \qquad \dot{h} = v_h, \tag{12}$$

$$\dot{v}_x = \frac{F_{D,x} + T\cos(\theta+\delta)}{m}, \qquad \dot{v}_h = -g(h) + \frac{F_{D,h} + T\sin(\theta+\delta)}{m}, \tag{13}$$

$$\dot{m} = -\frac{T}{g_0 I_{sp}}, \qquad \dot{\theta} = \omega, \tag{14}$$

$$I_d\dot{\omega} = M_a - \frac{L}{2}T\sin\delta, \qquad I_d = \frac{m_d(3R_b^2 + L^2)}{12}. \tag{15}$$

Thrust acts at the tail, $-(L/2)\mathbf{e}$, which gives the negative sign in the torque. The dry body is a uniform cylinder, and the remaining propellant is a point mass at the COM, so it adds no pitch inertia and I_d is constant. Propellant spread through real tanks would make the inertia change during the burn and add exhaust angular-momentum terms; both are neglected.

5.2 Spherical-Earth Correction

For the reference calculation we replace the flat geometry with a nonrotating spherical Earth of radius R_E. Write $r = R_E + h$ and $s = R_E\phi$, where s is surface arc distance. Velocities v_x and v_h are the physical components along the local horizontal and radial directions, which turn as the booster moves downrange. The state keeps its seven components, with x replaced by s. Let $a_x = F_x/m$ and $a_h = F_h/m$ denote the non-gravitational accelerations. Differentiating the moving basis gives

$$\dot{s} = \frac{R_E}{r} v_x, \qquad \dot{h} = v_h, \tag{16}$$

$$\dot{v}_x = a_x - \frac{v_x v_h}{r}, \qquad \dot{v}_h = a_h - g(h) + \frac{v_x^2}{r}. \tag{17}$$

The last terms in the velocity equations are the curvature accelerations, and the factor R_E/r converts horizontal speed at altitude into distance along the surface.

We keep $\omega = \dot{\theta}$ as the pitch rate relative to the local horizontal. Because the local horizontal rotates at $-v_x/r$ in the flight plane, the inertial body rate is $\omega_I = \omega - v_x/r$, and the strip velocity in Eq. (8) becomes $u_n + \omega_I \xi_i$. The angular equation is then

$$\dot{\omega} = \frac{M_a + M_T}{I_d} + \frac{a_x}{r} - \frac{2 v_x v_h}{r^2}, \tag{18}$$

where the last two terms come from differentiating v_x/r. With them, a torque-free body keeps a fixed inertial attitude while its pitch measured from the local horizontal changes as it travels downrange.

For contact with the sea-level deck, the longitudinal section of the cylinder is a rectangle in the flight plane. Its point nearest the Earth's center has body coordinates

$$a_* = \mathrm{clip}(-r\sin\theta, -L/2, L/2), \qquad b_* = \mathrm{clip}(-r\cos\theta, -R_b, R_b), \tag{19}$$

and first contact occurs when

$$f_s = \sqrt{(a_*\cos\theta - b_*\sin\theta)^2 + (r + a_*\sin\theta + b_*\cos\theta)^2} - R_E - h_{deck} = 0. \tag{20}$$

The ship is a fixed point on the surface, given by its arc coordinate. Within the 5 m position tolerance the local vertical turns by less than 10^{-6} rad, so the deck at the contact point is level in the local frame.

5.3 Initial Conditions and Contact Geometry

For each selected initial flight-path angle,

$$x_0 = 0, \quad h_0 = 70000 \text{ m}, \quad (v_{x,0}, v_{h,0}) = \frac{6000}{3.6}(\cos\gamma_0, \sin\gamma_0), \quad m_0 = m_d + m_{p,0}. \tag{21}$$

The reference uses $\gamma_0 = \theta_0 = 15^\circ$ and $\omega_0 = 0$. The separation propellant $m_{p,0}$ is an unknown found by the optimization of Section 6.

In the flat-Earth model, the cylinder extends $(L/2)|\sin\theta| + R_b|\cos\theta|$ below its COM. First deck contact occurs when

$$f(\mathbf{y}) = h - h_{deck} - \frac{L}{2}|\sin\theta| - R_b|\cos\theta| = 0. \tag{22}$$

The spherical model uses Eq. (20) instead. An upright booster touches the deck when $h - h_{deck} = L/2$. The contact condition only fixes the moment of touchdown; speed, attitude, rotation rate, reserve, and position relative to the ship are checked at that moment.

5.4 Analytic Baselines and Scale Estimates

For constant gravity, zero thrust, and zero drag,

$$h(t) = h_0 + v_{h,0}t - \tfrac{1}{2}g_0t^2, \qquad x(t) = v_{x,0}t. \tag{23}$$

At 15°, $(v_{x,0}, v_{h,0}) = (1609.88, 431.37)$ m/s. The constant-g_0 apex estimate is 79.49 km after 43.99 s. The integrated spherical trajectory, with drag and variable gravity, reaches 80.1 km. In the variable-gravity vertical vacuum limit, the conserved specific energy is

$$E = \tfrac{1}{2}v_h^2 - \frac{g_0R_E^2}{R_E + h}. \tag{24}$$

Without opposing external acceleration in the thrust direction, integrating $dv = -c\,dm/m$ yields

$$\Delta v = c\ln(m_0/m_f). \tag{25}$$

For 210 t initially and 135 t finally, this gives 1429.86 m/s. This is the ideal velocity change the propellant can provide. The velocity change actually needed differs, because drag removes speed for free while gravity and steering add to the requirement.

With mass frozen, no drag, and vertical maximum thrust, a useful stopping-distance estimate is

$$d_{stop} \simeq \frac{v_h^2}{2(T_{max}/m - g_0)}, \qquad T_{max} > mg_0. \tag{26}$$

For 180 t and a 90 m/s descent it gives 674.5 m, ignoring mass loss and drag. The thrust-to-weight ratio at full thrust grows from 1.382 at 210 t to 2.232 at dry mass, so the changing mass must be part of the dynamics.

5.5 Numerical Integration and Local Powered-Braking Check

A fixed-step fourth-order Runge–Kutta integrator advances the coupled equations. The step shrinks near the deck, and contact is located by linear interpolation across the sign change of Eq. (22), so the event time is less accurate than the integration itself. A propellant guard stops the run before a step would take the mass below the dry mass; it prevents negative propellant but does not find the exact depletion time.

To test the powered and rotational parts of the model on their own, we start a local case at $h = 1200$ m, $v_h = -90$ m/s, $v_x = 0$, and $m = 180000$ kg. Define height above upright contact $z = h - L/2$ and a descent-speed reference

$$v_d = -\min\{90, \sqrt{2k_z\max(z, 0.01) + v_c^2}\}, \quad k_z = 1.5\ \text{m/s}^2, \quad v_c = 1\ \text{m/s}. \tag{27}$$

A vertical acceleration command $a_c = k_v(v_d - v_h)$, $k_v = 4\ \text{s}^{-1}$, supplies the throttle request $[m(g_0 + a_c) - F_{D,h}]/T_{max}$, clipped to the permitted operating range. This test controller acts only along the vertical and uses g_0 for gravity. Pitch control requests

$$M_{req} = I_d[k_p(\pi/2 - \theta) - k_d\omega] - M_a, \quad \delta = \arcsin\left[\text{clip}\left(-\frac{M_{req}}{(L/2)T}, -1, 1\right)\right], \tag{28}$$

where $k_p = 0.5\ \text{s}^{-2}$, $k_d = 1.2\ \text{s}^{-1}$, followed by gimbal clipping. Without saturation this gives the stable error equation $\ddot{e} + k_d\dot{e} + k_pe = 0$. Saturation and engine-off flight break this linear argument, which is why the full trajectory is checked separately.

At a 0.005 s step the nominal case contacts after 35.97505 s with $v_h = -1.50291$ m/s and consumes 22679.12 kg. A separate test with initial pitch 89° and initial rate 0.005 rad/s ends with approximately zero pitch error and $v_x = 0.4671$ m/s. Its horizontal drift is 16.56 m, exceeding the 5 m fixed-ship position tolerance if the ship remains at $x = 0$. The attitude recovers, but the booster misses a fixed ship; reaching the ship is the task of the full-flight optimization in Section 6. Figure 2 shows both runs.

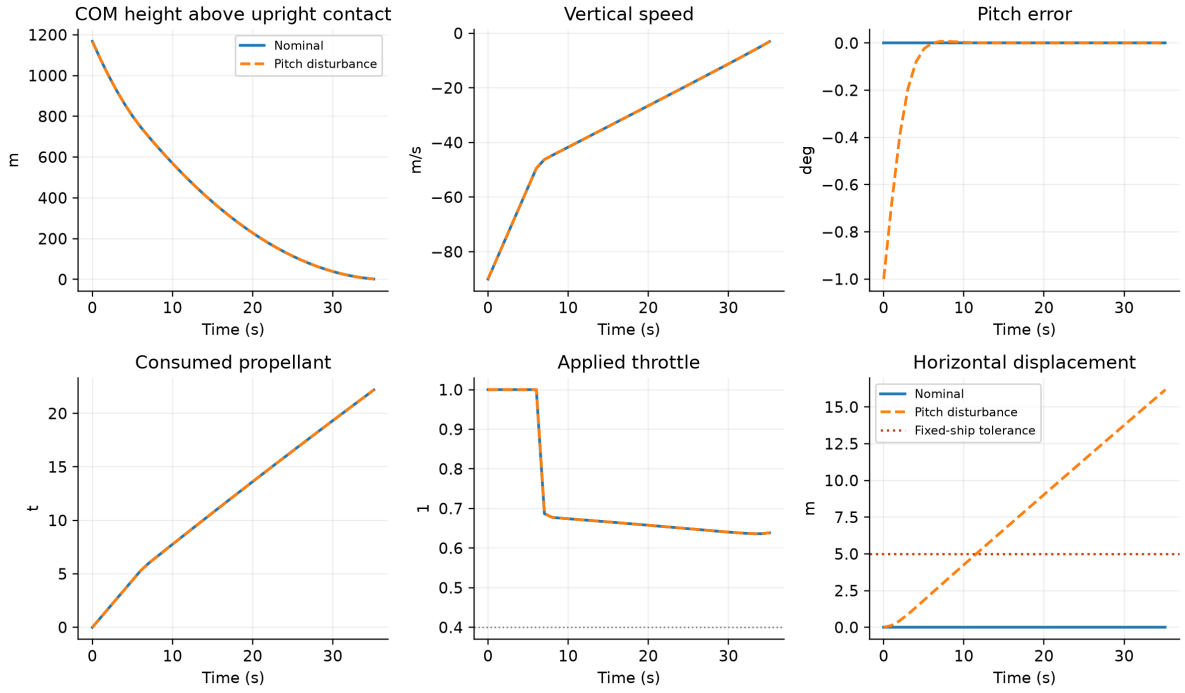


Figure 2: Local powered-braking test. The disturbed case starts at 89° pitch with a 0.005 rad/s pitch rate. The 5 m line shows that recovering the attitude does not by itself bring the booster to the ship.

6 Landing Strategy and Propellant Estimation

6.1 Spherical-Earth Reference Solution

Starting from the planar solution, we optimize the spherical equations with the same dry mass, propulsion model, touchdown limits, and 15° flight-path angle at separation. The best solution we found needs 25.97 t of propellant at separation, 5.77 t of methane and 20.20 t of liquid oxygen, including the 5 t reserve. The booster coasts until 181.3 s, lights its engine at 6.2 km altitude, and touches down at 213.25 s on a ship 284.12 km downrange along the surface.

In the replay at a 0.00125 s step, the booster lands 2.7 mm from the target, moving 0.0028 m/s sideways and 1.507 m/s downward, tilted 0.00034° from vertical and rotating at 0.0129°/s. It burns 20.89 t and keeps 5.08 t. Dynamic pressure peaks at 292.9 kPa and specific force at 7.84 g_0.

Speed budget. In still air, projecting the translational equations along the velocity gives $\dot{U} = \hat{\mathbf{v}} \cdot [(\mathbf{F}_a + \mathbf{T})/m - g(h)\hat{\mathbf{e}}_h]$, where $\hat{\mathbf{v}}$ is the velocity direction and $\hat{\mathbf{e}}_h$ is the

outward radial direction. The curvature terms cancel in this projection. Integrating these contributions along the replay, aerodynamics removes about 1148 m/s during the coast and gravity adds a net 383 m/s, leaving 901 m/s at ignition. During the burn, aerodynamics removes another 679 m/s and thrust removes 414 m/s, while gravity adds 193 m/s. The remaining speed is about 1.51 m/s. The integral of thrust acceleration magnitude is $\int T/m\,dt = 465$ m/s, consistent with Eq. (25). Its projection along the velocity accounts for the 414 m/s contribution. Aerodynamic braking therefore supplies most of the speed reduction. The 20.89 t landing burn provides additional braking and brings the booster upright at contact.

Optimization meshes of 0.85, 0.55, and 0.40 s give 26.11, 26.05, 26.04 t, respectively. Three further optimization rounds on the 0.40 s mesh give 26.02, 25.97, 26.01 t; we keep the lowest. Their spread of about 0.05 t shows how precisely the local optimum is resolved. Every reported solution passes the replay at 0.005 and 0.0025 s, and the selected one also at 0.00125 s. The spherical result is 0.26 t lighter than the 26.23 t planar design. Both the trajectory and the controls were reoptimized, so this is the difference between two designs, not a correction applied to one.

Over a 100 s vacuum coast, the integrator conserves specific energy to 7.5×10^{-9} J/kg and angular momentum to a relative error of 6.7×10^{-16}. Without torque, the inertial attitude stays constant to 5.9×10^{-16} rad. At 32 sampled states, the derivatives used by the optimizer agree with those of the replay code to within 4.0×10^{-16} after scaling. These tests confirm the signs of the moving-frame terms and the consistency of the two implementations.

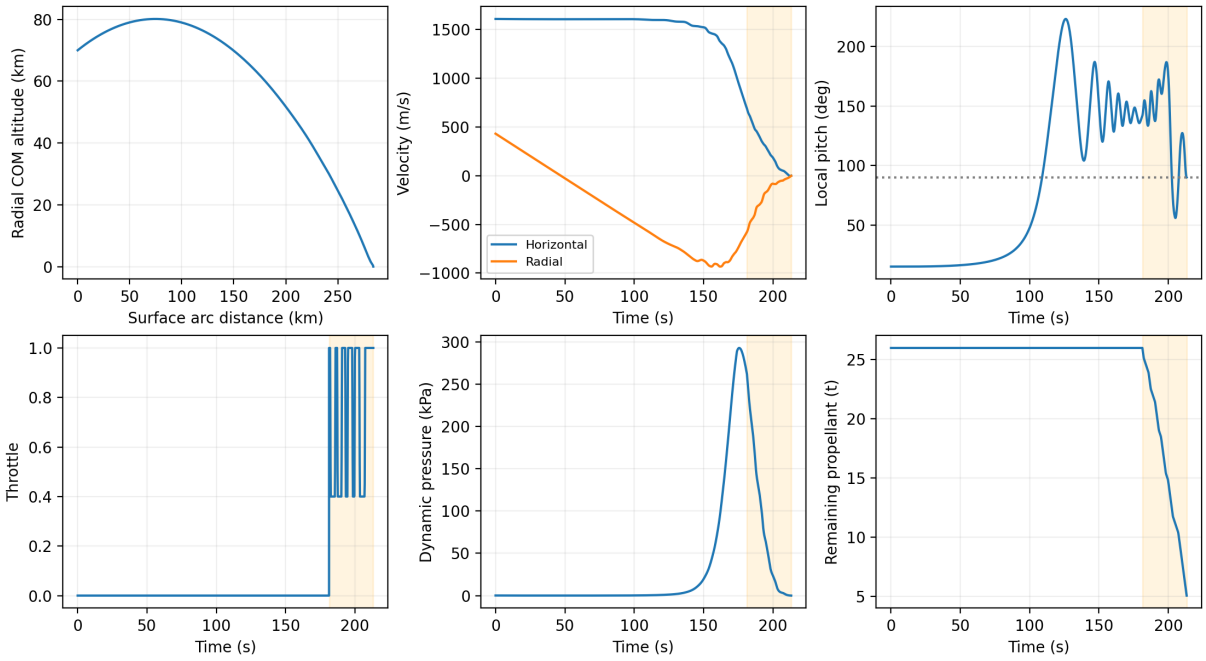


Figure 3: Spherical-Earth reference trajectory. Downrange position is distance along the surface, altitude is radial, and pitch is measured from the local horizontal. Shading marks the landing burn.

6.2 Search and Minimum-Propellant Formulation

We used a feedback controller on the planar model to generate starting trajectories. It varied the ignition altitude, initial inventory, pitch bias, and descent-speed profile. During the burn, the pitch target changes from a biased retrograde direction to upright with lateral-velocity feedback. The throttle tracks a decreasing descent speed, and the gimbal supplies a pitch-tracking moment.

The optimization minimizes $m(0) - m_d$ over phase durations, throttle and gimbal commands, and trajectory states. All seven states are continuous across phase boundaries. Coast phases have zero thrust. Powered phases obey $0.4 \le u_T \le 1$ and $|\delta| \le 10^\circ$. The ship position is free during optimization and fixed during replay. The terminal targets are deck contact, $v_x = 0$, $v_h = -1.5\,\mathrm{m/s}$, upright attitude, and zero pitch rate. The optimizer includes 75 kg above the 5 t usable reserve to allow for integration differences in the replay.

Hermite–Simpson direct collocation initializes the planar solutions [5]. RK4 multiple shooting refines them and reoptimizes the spherical design. Throttle and gimbal are linearly interpolated within each phase. Each shooting interval uses eight RK4 substeps, with a local trust region on phase duration and pitch. CasADi supplies derivatives to IPOPT [6, 7]. We then integrate each accepted control schedule independently to first contact and check the touchdown limits at the selected ship position.

6.3 Planar Comparison and Inventory Accounting

At 15° the planar model gives 26.23 t and a ship location of 278.31 km with one ignition. The lowest verified planar two-ignition result requires 27.61 t and drives its first pulse to the 0.001 s numerical duration floor. A feedback trajectory with a separate 1.5 s entry burn lands with 38.65 t. The one-ignition planar result seeds the spherical reference in Section 6.1.

We account for the separation inventory as

$$m_{p,0} = m_{\text{burned}} + m_{\text{reserve}} + m_{\text{residual}} + m_{\text{surplus}}. \tag{29}$$

The nominal spherical solution burns 20.89 t and retains 5.08 t, of which 5 t is the planned usable reserve and about 0.08 t is surplus. The unusable residual is set to zero.

6.4 Dependence on the Separation Direction

The problem gives the speed at separation but not its direction. That direction is set by the ascent, not by the recovery plan, so it is an input to the landing design and not a variable to optimize. A first stage normally separates while the vehicle is still climbing, so we take 15° above horizontal as the reference and compare it with 0° and 30°. This choice is our assumption. In each case the body axis starts aligned with the velocity, the propulsion model, touchdown limits, and 5 t reserve are unchanged, and the ship is placed separately.

The tabulated standard atmosphere ends at 84.852 km geopotential altitude. The earlier planar study stopped any trajectory that climbed higher, which forced its 30° design to brake during the ascent. Here we continue the atmosphere above that altitude at a constant 186.95 K. The density is $7 \times 10^{-6}\,\mathrm{kg/m^3}$ at the junction and about $1.5 \times 10^{-7}\,\mathrm{kg/m^3}$ at 108 km, so drag up there is negligible. The real upper atmosphere is not

isothermal, but at these densities the difference has little effect on the trajectory. The 15° reference reaches only 80.1 km and is unaffected.

Table 1: Accepted strategies for three separation angles. Ship position is surface arc distance from the separation point. Inventories include the 5 t reserve. The planar values stopped the atmosphere at 84.852 km geopotential altitude.

Angle	Apex (km)	Ship (km)	Inventory (t)	Planar ship (km)	Planar inventory (t)
0°	70.0	199.26	24.32	196.00	24.71
15°	80.1	284.12	25.97	278.31	26.23
30°	107.7	348.98	32.46	260.00	67.00

All three spherical designs coast to a single landing burn. The 0° design never climbs, lands closest, and needs 1.65 t less than the reference. At 30° the booster coasts up to 107.7 km and starts its landing burn at 25.9 km. It needs 32.46 t, about half of the 67.00 t planar value, because without the artificial altitude limit there is no need to brake on the way up. Over the range of separation directions considered, the inventory changes by about 8 t, more than for any engine or aerodynamic parameter in Section 7.

7 Sensitivity and Flight Loads

Several quantities that set the propellant requirement are not given in the problem. The separation direction was treated in Section 6.4. Here we vary the effective specific impulse and the crossflow drag coefficient on the spherical model, keeping the 15° separation and the 5 t reserve. Each case is a separate design: the parameter is assumed known in advance, and the ship is placed for that case. Whether a booster could recover from an unexpected parameter error with the ship already in position is a different question, touched on in Section 7.5.

7.1 Dependence of the Propellant Budget on Assumptions

7.1.1 Effective Specific Impulse

We compare effective specific impulses of 300, 330, and 360 s, with the thrust, throttle range, gimbal limit, aerodynamic coefficients, and mixture ratio unchanged. Each case starts from the spherical reference and reoptimizes the inventory, burn timing, controls, and ship position for a single landing burn. All three solutions meet the touchdown limits in replays at 0.005 and 0.0025 s; Table 2 lists the finer replay.

Table 2: Effective-I_{sp} comparison on the spherical model. Each design reoptimizes the inventory, controls, and ship position. Masses in tonnes, rounded independently. The last column is the earlier planar result.

I_{sp} (s)	Inventory	Consumed	Retained	Methane	Oxygen	Planar
300	28.48	23.41	5.08	6.33	22.15	28.78
330	25.97	20.89	5.08	5.77	20.20	26.23
360	23.98	18.90	5.07	5.33	18.65	24.20

Table 3: Crossflow-coefficient comparison on the spherical model, axial coefficient fixed at 0.8. The last column is the earlier planar result; a dash marks a case without a planar solution.

C_n	Inventory (t)	Peak q (kPa)	Peak specific force (g_0)	Planar (t)
0.8	25.94	295.5	8.70	–
1.0	25.96	293.7	8.21	–
1.2	25.97	292.9	7.84	26.23
1.6	25.49	290.2	7.32	27.13

Relative to 330 s, the inventory rises by 2.52 t (9.7%) at 300 s and falls by 1.99 t (7.7%) at 360 s. Scaling the burned mass inversely with specific impulse would give 28.06 and 24.23 t instead. The difference arises because the inventory also carries the fixed 5 t reserve and because a heavier booster descends differently, so the whole trajectory has to be replanned. The planar values in the last column follow the same trend and lie 0.2–0.3 t higher. Holding thrust and mixture ratio fixed while changing specific impulse is a simplification; a real engine with different efficiency would differ in other respects too.

7.1.2 Crossflow Drag

The crossflow coefficient affects aerodynamic braking, the pitching moment, and rotational damping at the same time. With the axial coefficient fixed at 0.8, Table 3 compares $C_n = 0.8$, 1.0, 1.2, and 1.6 using the same optimization and touchdown limits. The $C_n = 0.8$ case did not converge when started directly from the reference, so we started it from the $C_n = 1.0$ solution.

Between $C_n = 0.8$ and 1.2 the inventory changes by less than 0.03 t, which is smaller than the 0.05 t spread between repeated optimizations of one case. At $C_n = 1.6$ it is 0.47 t lower, and the peak specific force falls from $7.84\,g_0$ to $7.32\,g_0$ because more of the braking is done by the air. Over this range, crossflow drag has little effect on the propellant answer. The earlier planar study gave 27.13 t at $C_n = 1.6$ and found no solution at 0.8; those runs used fewer optimization rounds and a 140 s solver time limit, and the spherical results replace them.

7.2 Loads Along the Nominal Descent

The loads on the 25.97 t spherical trajectory vary throughout the descent (Table 4). Dynamic pressure is $q = \rho U^2/2$, where U is the speed relative to the air [8]. We express the magnitude of the specific force in units of g_0 as $\|\mathbf{F}_a + \mathbf{T}\|/(mg_0)$.

Dynamic pressure peaks before the engine lights at 181.32 s, while the specific force peaks during the burn. Near that second peak the body axis is about 20° from the direction of the oncoming air, which produces a large side force. The attitude therefore matters as much as the speed and the air density. Because structural and thermal capacity are not modeled, the table states what a booster flying this trajectory would have to withstand; it does not show that it can.

Table 4: Peak loads on the spherical reference trajectory, from the replay at a 0.0025 s step. Height is the radial altitude of the center of mass. The planar design gave 297.3 kPa, 7.70 g_0, and 9.62 MN.

Quantity	Time (s)	Height (km)	Peak
Dynamic pressure	175.725	9.834	292.9 kPa
Specific-force magnitude	187.413	3.343	7.84 g_0
Normal aerodynamic force	187.523	3.302	9.98 MN

7.3 Aerodynamic Heating Estimate

To gauge the heating, we apply the Sutton–Graves correlation for the convective heat flux at a stagnation point in air [9],

$$\dot{q}_s = k\sqrt{\frac{\rho}{R_n}}\,U^3, \qquad k = 1.7415 \times 10^{-4}\ \mathrm{kg}^{1/2}\,\mathrm{m}^{-1}, \tag{30}$$

along the reference trajectory, where R_n is the effective nose radius of the windward surface. The booster changes attitude during descent and meets the air partly tail first, so we evaluate two representative radii, 1 m for a rounded edge and 3.5 m for the body radius. The peak occurs at 168.7 s, at 15.5 km altitude and 1500 m/s, before the dynamic-pressure peak. It is 249 kW/m^2 for $R_n = 1$ m and 133 kW/m^2 for $R_n = 3.5$ m. Integrating the estimated stagnation-point heat flux over flight gives incident convective heat loads of 8.6 and 4.6 MJ/m^2, respectively. The calculation omits radiation, conduction, and the surface temperature response. The correlation and the assumed effective radii support an order-of-magnitude estimate of the thermal environment.

7.4 Cost of Limiting Dynamic Pressure

The 293 kPa peak occurs while the engine is off, so a later correction cannot reduce it. To see what a lower structural limit would cost, we add the constraint $q \le q_{max}$ to the spherical optimization at every mesh point, with a 1% margin. Starting trajectories come from the feedback search with one extra braking burn high in the descent, followed by a second coast and the landing burn. A solution is accepted only if both replays meet every touchdown limit and the replayed peak stays at or below q_{max}. We have no sourced load limit for this vehicle, so the 200 and 150 kPa caps are examples.

Table 5: Dynamic-pressure caps for the 15° spherical case. Each row is the best accepted solution for that limit. Masses in tonnes.

Cap (kPa)	Status	Burns	Inventory	Entry burn	Landing burn	Peak q (kPa)
none	optimized	1	25.97	–	20.89	292.9
200	optimized	2	37.30	13.50	18.72	198.0
150	optimized	2	50.17	27.08	18.05	148.7

A 200 kPa cap raises the separation inventory from 25.97 t to 37.30 t. The added propellant goes into a 38 s entry burn starting at 43.2 km altitude, while the landing burn still uses about 19 t. A 150 kPa cap needs 50.17 t, with a 77 s entry burn starting

at 62.7 km altitude. Lowering the limit from 200 to 150 kPa costs another 12.9 t. These solutions pass the same replay checks as the reference. They were still improving when the optimization was stopped, so they are upper estimates of the minimum.

Across the modeled mass range, full thrust contributes roughly 1.5–2.2 g_0 of acceleration before projection onto the flight direction. In the two capped trajectories, the long entry burn reduces speed before the booster reaches denser air and accounts for most of the added propellant.

7.5 A Limited Guidance Extension

If the precomputed commands are flown unchanged, wind or an error in the initial attitude generally makes the booster miss the ship. To test in-flight correction, we built a 30 t feedback design on the planar model. With exact knowledge of the state and instantaneous actuators, it recovered steady-wind errors of -0.1, -0.05, 0, $+0.05$, and $+0.1$ m/s and landed on the same ship. Its nominal peak specific force was about 8.13 g_0, higher than in the reference. This shows that feedback can absorb small errors in these five cases. It has not been repeated on the spherical model, and its 30 t trial inventory is unrelated to the propellant estimate. The replanning runs, wind-error sweeps, and controller settings are kept in the repository.

7.6 Summary of the Parameter Studies

Each study changes one parameter in a separate design with its own trajectory and ship position, and every inventory includes the 5 t reserve. On the spherical model, the separation direction moves the answer by about 8 t over 0°–30°, the effective specific impulse by -2.0 to $+2.5$ t over 360–300 s, and the crossflow coefficient by less than 0.5 t. A dynamic-pressure limit, if one is imposed, outweighs all of these: a 200 kPa limit raises the inventory to about 37.30 t. The reference answer in Section 6.1 has no such limit.

8 Model Evaluation

8.1 Internal Verification and Numerical Convergence

The checks below compare the implementation with known limiting solutions; they are not a comparison with flight data. Standard-atmosphere base pressures at 0, 11, 20, 32, and 47 km geopotential altitude agree with rounded reference values to within 4.8×10^{-6} relative error, and queries above the tabulated range raise an error unless the isothermal continuation is enabled. A vertical vacuum coast conserves specific mechanical energy to 7.5×10^{-9} J/kg over 20 s. A constant-thrust mass history and a horizontal ideal-rocket velocity agree with their analytic forms to 7.6×10^{-10} kg and 2.9×10^{-14} m/s.

We also check the pure axial and pure crossflow force limits, the static pressure-center torque, torque-free rotation, the signs of damping and gimbal torque, and the energy dissipation relative to air. For pure centered rotation, Eq. (9) gives the exact torque. The midpoint-strip errors are 2%, 0.5%, 0.125%, 0.03125%, and 0.0078125% for 10, 20, 40, 80, and 160 strips, so the default 20 strips carry a 0.5% quadrature error in this case, far below the uncertainty of the aerodynamic coefficients themselves.

A four-second smooth unpowered trajectory is compared with a run at 0.00125 s using the same equations. The position difference falls from 9.75×10^{-7} m at a 0.08 s

step to 2.76×10^{-10} m at 0.01 s, consistent with fourth-order convergence. Because the force laws contain absolute values and the controllers clip their commands, this order is expected only on smooth segments. Figure 4 shows the three error sources separately. Between the two finest steps, the consumed mass changes by 0.00384 kg and the contact

Table 6: Convergence of the local powered-braking case of Section 5. Every row meets the speed, attitude, and reserve limits.

Step (s)	Contact time (s)	v_h (m/s)	Consumed (kg)
0.080	35.975945	-1.502940	22679.544
0.040	35.975286	-1.503756	22679.191
0.020	35.975097	-1.502978	22679.139
0.010	35.975063	-1.502986	22679.123
0.005	35.975047	-1.502909	22679.119

time by 1.60×10^{-5} s. The contact speed does not change monotonically, as expected from the linear interpolation used to locate the contact event. A separate test confirms that the propellant guard stops before the mass drops below the dry mass. These checks

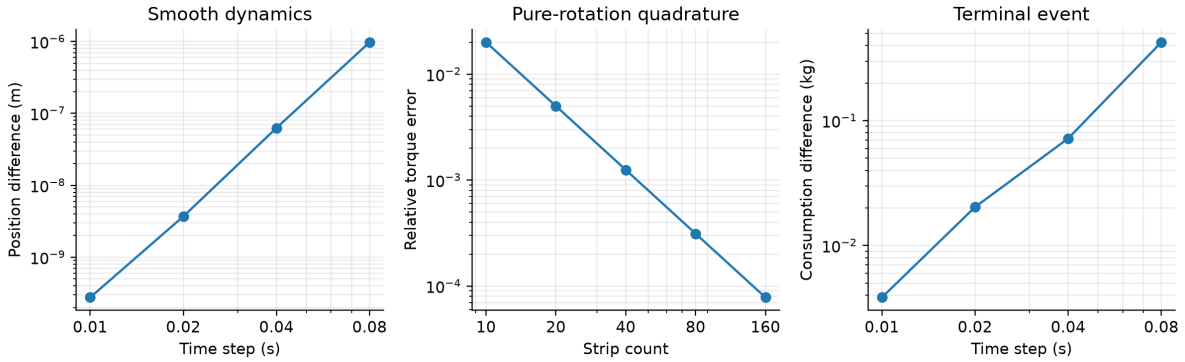


Figure 4: Numerical error sources. Left: position difference of a smooth trajectory against a finer run of the same model. Middle: strip-torque error against the exact integral. Right: difference in consumed mass at contact relative to the 0.005 s run.

are reproduced with `node src/verify-p1.mjs`. The spherical-Earth checks are reported with the spherical results in Section 6.1.

8.2 Strengths and Weaknesses

The main strength of the model is that the trajectory, the attitude, and the propellant budget come from one calculation. The landing state and the fuel figure therefore describe the same flight. The model includes changing mass, pitch dynamics with aerodynamic damping, Earth curvature, and contact between the tilted cylinder and the curved sea surface, and every reported solution is checked by an independent replay at two time steps.

Its weaknesses are the assumed engine and aerodynamic parameters, which are not fitted to a real vehicle; ideal actuators without rate limits; a simplified mass distribution; a rigid structure with no structural assessment and only an order-of-magnitude heating estimate; the neglect of Earth rotation and out-of-plane motion; and solutions that are

local optima. Agreement within the model says nothing about the accuracy of these assumptions. Section 7 shows how much some of them move the answer.

9 Discussion and Conclusions

9.1 Attitude Stability and Vertical Touchdown

The reference trajectory meets the position, speed, tilt, and pitch-rate limits at the same instant of contact. All four are needed: an upright booster that is still drifting or rotating has not landed under control. While the engine is off, nothing in the model controls the attitude; during the burn, the available correction is limited by thrust and gimbal travel. The nominal touchdown result holds under these limits.

9.2 Loads and Scope

The booster is treated as a rigid body. Material failure, structural sizing, and thermal protection are outside this study, so the dynamic pressure, aerodynamic moments, specific force, and heating in Sections 7.2–7.3 are requirements that a real vehicle would have to meet, not pass-or-fail results. The touchdown limits concern position, relative velocity, attitude, rotation rate, and propellant reserve. The engine's thrust and gimbal ranges are enforced, but their rates of change are not.

9.3 Earth Curvature and Rotation

Over the 278 km range of the flat-Earth design, the flat ground plane departs from the sea surface by about 6.1 km, far more than the 5 m landing tolerance. We therefore solve the spherical equations, Eqs. (16)–(20), in both the optimization and the replay, rather than correcting the planar landing point afterwards. The two optimal ship positions differ by 5.8 km.

Earth rotation is neglected. Its effect depends on the latitude and heading of the flight, which the problem does not give, and including it would require a three-dimensional trajectory.

9.4 Ship Placement, Wind, and Deck Motion

Placing the ship downrange, under the descent path, removes the need to fly back and allows a single landing burn. Once the ship is in place, however, the booster has to reach that point, so any wind or other error not known in advance must be corrected in flight. The ship here is stationary with a level deck. A moving deck would change the target position, the relative velocity at contact, and the deck orientation.

9.5 Conclusions and Practical Recommendations

For the spherical-Earth reference with a 15° separation direction, a coast followed by one landing burn needs 25.97 t of propellant at separation, the lowest value we found. At an oxygen-to-methane mass ratio of 3.5 this is 5.77 t of methane and 20.20 t of liquid oxygen, and it includes the 5 t usable reserve. The ship is placed 284.12 km downrange

along the surface, and the trajectory meets all contact limits. The flat-Earth version of the same design needed 26.23 t, with the ship at 278.31 km.

The answer depends most on inputs that the problem leaves open. The separation direction, which the ascent mission fixes, moves the requirement from 24.32 t at 0° to 32.46 t at 30°, and the ship moves from 199 to 349 km downrange. Effective specific impulse between 300 and 360 s gives 28.48 to 23.98 t, whereas crossflow drag between $C_n = 0.8$ and 1.6 changes the result by less than 0.5 t. In practice, the separation direction and the engine efficiency should be fixed first, because together they determine both the propellant load and where the ship must wait.

The reference descent reaches 293 kPa dynamic pressure before ignition. With a separate entry burn, we found feasible inventories of about 37.30 t at a 200 kPa cap and 50.17 t at a 150 kPa cap (Section 7.4). These are upper estimates within the tested strategy family. All figures assume the reference engine and drag parameters, a nonrotating Earth, a stationary ship, ideal actuators, and a rigid structure.

References

[1] Blue Origin. BE-4. Manufacturer webpage, accessed November 7, 2026. https://www.blueorigin.com/es-MX/engines/be-4.

[2] F. E. Gowen and E. W. Perkins. Drag of Circular Cylinders for a Wide Range of Reynolds Numbers and Mach Numbers. NACA-TN-2960, 1953.

[3] NOAA, NASA, and USAF. U.S. Standard Atmosphere, 1976. NASA-TM-X-74335, 1976.

[4] Mars Sample Return Mission Utilizing In-Situ Propellant Production. NASA NTRS record 19950021840. Methane/oxygen performance discussion, Fig. 2.

[5] M. Kelly. An Introduction to Trajectory Optimization: How to Do Your Own Direct Collocation. SIAM Review 59(4), 849–904, 2017. https://doi.org/10.1137/16M1062569.

[6] J. A. E. Andersson et al. CasADi: A Software Framework for Nonlinear Optimization and Optimal Control. Mathematical Programming Computation 11, 1–36, 2019. https://doi.org/10.1007/s12532-018-0139-4.

[7] A. Wächter and L. T. Biegler. On the Implementation of an Interior-Point Filter Line-Search Algorithm for Large-Scale Nonlinear Programming. Mathematical Programming 106, 25–57, 2006. https://doi.org/10.1007/s10107-004-0559-y.

[8] NASA Glenn Research Center. Dynamic Pressure. Accessed November 7, 2026. NASA Glenn webpage.

[9] K. Sutton and R. A. Graves, Jr. A General Stagnation-Point Convective Heating Equation for Arbitrary Gas Mixtures. NASA-TR-R-376, 1971. https://ntrs.nasa.gov/citations/19720003329.

[10] The University Physics Competition. Problem A: Landing a First-Stage Rocket Booster, 2026. Official problem statement.

A Reproducibility and Source Code

The corrected reference parameters are in `config/spherical.json`. The planar comparison uses `config/reference.json`.
The touchdown conventions are stored in `config/p0-contract.json`. Spherical optimization and replay are reproduced with `src/spherical-optimize.py` and `src/spherical-report.py`. The accepted controls and finer-step replay are archived in `results/spherical/selected.json`. The selected complete trajectories and inventories are in `results/p2-p0-v1/`; solver commands and dependencies are documented in `docs/p2-landing-strategy.md` and `requirements-p2.txt`.
The planar efficiency, crossflow, and nominal-load records are in `results/p3/propulsion/`, `results/p3/drag/`, and `results/p3/safety/`. Their drivers are `src/p3-propulsion.py`, `src/p3-drag.py`, and `src/p3-exposure.mjs`, with reproduction commands in the corresponding documentation. The additional fixed-ship replanning and feedback experiments remain in `results/p3/replanned/`, `results/p3/wind-estimation.json`, and `results/p3/tracking/`. These supplementary experiments are distinct from the nominal design used to answer the propellant question.
The spherical separation-angle, parameter, dynamic-pressure-cap, and heating studies use the following files.

- Results: `results/spherical/angle/`, `results/spherical/sensitivity/`, `results/spherical/qcap/`, and `results/spherical/heating.json`, collected in `results/spherical/summary.json` by `src/spherical-summary.py`.
- Separation angle: `src/spherical-angle-search.mjs` and `src/spherical-angle-optimize.py`.
- Parameter studies: `src/spherical-sensitivity.py`.
- Dynamic-pressure caps: `src/spherical-q-search.mjs` and `src/spherical-q-optimize.py`.
- Heating estimate: `src/spherical-heating.mjs`.
- Tables and generated text: `src/spherical-tables.py` and `src/spherical-report.py`.

Commands are listed in `docs/p5-task-list.md`.

A.1 Core Source Listings

The listings are drawn directly from the source files. They show the atmospheric model, force and contact calculations, constrained optimization, and independent replay used for the spherical reference. The driver scripts and case settings are identified above.

Listing 1: Standard atmosphere and isothermal extension

```
const G=9.80665,R=6356766,Ra=287.05287;
// COESA lower-atmosphere lapse rates; geopotential layer boundaries (m).
const H=[0,11000,20000,32000,47000,51000,71000,84852];
const lapse=[-.0065,0,.001,.0028,0,-.0028,-.002];
const temps=[288.15],press=[101325];
for(let i=0;i<7;i++){
 const d=H[i+1]-H[i],b=lapse[i],t=temps[i]+b*d;
 temps.push(t);press.push(b===0?press[i]*Math.exp(-G*d/(Ra*temps[i])):press[i]*(t/
  ↪ temps[i])**(-G/(Ra*b)));
}
// Optional isothermal continuation above 84.852 km geopotential. Density there is
// below 7e-6 kg/m^3, so the extension only prevents an artificial altitude cap.
export const UPPER_LIMIT_GEOPOTENTIAL_M=200000;
export function atmosphere(z,extended=false){
 const h=R*z/(R+z);
 if(h<0||h>(extended?UPPER_LIMIT_GEOPOTENTIAL_M:H.at(-1)))throw Error('Atmosphere
  ↪ altitude outside implemented range');
 if(h>H.at(-1)){
  const t=temps.at(-1),p=press.at(-1)*Math.exp(-G*(h-H.at(-1))/(Ra*t));
  return {temperatureK:t,pressurePa:p,density:p/(Ra*t)};
 }
 let i=0;while(i<6&&h>=H[i+1])i++;
 const d=h-H[i],b=lapse[i],t=temps[i]+b*d;
 const p=b===0?press[i]*Math.exp(-G*d/(Ra*temps[i])):press[i]*(t/temps[i])**(-G/(
  ↪ Ra*b));
 return {temperatureK:t,pressurePa:p,density:p/(Ra*t)};
}
```

Listing 2: Aerodynamic strip forces, rigid-body dynamics, RK4 and deck clearance

```
import {atmosphere} from './atmosphere.mjs';
export const g0=9.80665, rad=Math.PI/180;
export const clamp=(x,a,b)=>Math.max(a,Math.min(b,x));
export function inertia(p){
 // Dry body uniform cylinder; remaining propellant idealized at COM with
  ↪ negligible pitch inertia.
 return p.dryMassKg*(3*(p.diameterM/2)**2+p.lengthM**2)/12;
}
export function aero(y,p){
 const [x,h,vx,vz,m,theta,omega]=y;
 const e=[Math.cos(theta),Math.sin(theta)],n=[-e[1],e[0]];
 const air=atmosphere(h,p.upperAtmosphere==='isothermal'),u=[vx-p.windMps,vz];
 const axial=u[0]*e[0]+u[1]*e[1],normal=u[0]*n[0]+u[1]*n[1];
 const fa=-.5*air.density*p.axialCd*Math.PI*p.diameterM**2/4*axial*Math.abs(axial)
  ↪ ;
 let fn=0,torque=0;
 const ds=p.lengthM/p.strips;
 // Equivalent crossflow strip distribution shifted to represent uncertain CP
  ↪ offset.
 for(let i=0;i<p.strips;i++){
  const s=-p.lengthM/2+(i+.5)*ds+p.cpOffsetM;
```

```
  const inertialPitchRate = p.geometryModel === 'spherical' ? omega - vx / (6356766 + h) : omega;
  const local=normal+inertialPitchRate*s;
  const f=-.5*air.density*p.crossflowCd*p.diameterM*ds*local*Math.abs(local);
  fn+=f;torque+=s*f;
 }
 return {fx:fa*e[0]+fn*n[0],fz:fa*e[1]+fn*n[1],torque,fa,fn,q:.5*air.density*(u[0]**2+u[1]**2),mach:Math.hypot(...u)/Math.sqrt(1.4*287.05287*air.temperatureK)};
}
export function rhs(t,y,p,command){
 const [x,h,vx,vz,m,theta,omega]=y,a=aero(y,p);
 const c=command(t,y,a),on=c.throttle>0&&m>p.dryMassKg;
 const T=on?p.maxThrustN*clamp(c.throttle,p.minThrottle,1):0;
 const delta=clamp(c.gimbalRad||0,-p.gimbalLimitDeg*rad,p.gimbalLimitDeg*rad);
 // Tail at -L/2 along body axis: positive gimbal generates negative pitch torque.
 const torqueT=-p.lengthM/2*T*Math.sin(delta);
 const forceX = (a.fx + T * Math.cos(theta + delta)) / m;
 const forceH = (a.fz + T * Math.sin(theta + delta)) / m;
 const accelerationPitch = (a.torque + torqueT) / inertia(p);
 if (p.geometryModel === 'spherical') {
  const radius = 6356766 + h;
  return [6356766 / radius * vx, vz,
   forceX - vx * vz / radius,
   -g0 * (6356766 / radius) ** 2 + forceH + vx * vx / radius,
   -T / (p.ispS * g0), omega,
   accelerationPitch + forceX / radius - 2 * vx * vz / (radius * radius)];
 }
 return [vx,vz,forceX,-g0*(6356766/(6356766+h))**2+forceH,-T/(p.ispS*g0),omega,accelerationPitch];
}
export function rk4(t,y,dt,p,control){
 const add=(k,f)=>y.map((v,i)=>v+f*k[i]);
 const k1=rhs(t,y,p,control),k2=rhs(t+dt/2,add(k1,dt/2),p,control),k3=rhs(t+dt/2,add(k2,dt/2),p,control),k4=rhs(t+dt,add(k3,dt),p,control);
 return y.map((v,i)=>v+dt*(k1[i]+2*k2[i]+2*k3[i]+k4[i])/6);
}
export function clearance(y,p){
 if (p.geometryModel === 'spherical') {
  const radius = 6356766 + y[1];
  const sine = Math.sin(y[5]);
  const cosine = Math.cos(y[5]);
  const axial = clamp(-radius * sine, -p.lengthM / 2, p.lengthM / 2);
  const normal = clamp(-radius * cosine, -p.diameterM / 2, p.diameterM / 2);
  return Math.hypot(axial * cosine - normal * sine, radius + axial * sine + normal * cosine) - 6356766 - p.deckHeightM;
 }
 return y[1]-p.deckHeightM-p.lengthM/2*Math.abs(Math.sin(y[5]))-p.diameterM/2*Math.abs(Math.cos(y[5]));
}
```

Listing 3: Symbolic dynamics and constrained trajectory optimization

```
"""Optimize P0-v1 separation inventory, then replay schedules in P1 Node.js."""

import argparse
import json
from pathlib import Path
import subprocess
```

```
7
8  import casadi as ca
9  import numpy as np
10
11
12 ROOT = Path(__file__).resolve().parents[1]
13 SCALE = np.array([300_000, 80_000, 1_500, 1_000, 150_000, np.pi, 1])
14 R = 6_356_766.0
15 G0 = 9.80665
16 R_AIR = 287.05287
17 LAYERS = [0, 11_000, 20_000, 32_000, 47_000, 51_000, 71_000, 84_852]
18 LAPSES = [-0.0065, 0, 0.001, 0.0028, 0, -0.0028, -0.002]
19
20
21 def load(path):
22     return json.loads(path.read_text(encoding="utf-8-sig"))
23
24
25 def density(h, p):
26     """Symbolic transcription of atmosphere.mjs."""
27     geopotential = R * h / (R + h)
28     temperature, pressure = 288.15, 101325.0
29     rho = 0
30     for i, lapse in enumerate(LAPSES):
31         delta = geopotential - LAYERS[i]
32         temp = temperature + lapse * delta
33         if lapse:
34             # Keep unused branches differentiable outside their own layers.
35             pres = pressure * (ca.fmax(temp, 100) / temperature) ** (-G0 / (R_AIR
   ↪ * lapse))
36         else:
37             pres = pressure * ca.exp(-G0 * delta / (R_AIR * temperature))
38         layer_rho = pres / (R_AIR * ca.fmax(temp, 100))
39         rho = layer_rho if i == 0 else ca.if_else(geopotential >= LAYERS[i],
   ↪ layer_rho, rho)
40         width = LAYERS[i + 1] - LAYERS[i]
41         upper = temperature + lapse * width
42         pressure *= ((upper / temperature) ** (-G0 / (R_AIR * lapse)) if lapse
43                      else np.exp(-G0 * width / (R_AIR * temperature)))
44         temperature = upper
45     if p.get("upperAtmosphere") == "isothermal":
46         # Matches atmosphere.mjs: isothermal continuation above the last COESA
   ↪ layer.
47         upper_rho = pressure * ca.exp(-G0 * (geopotential - LAYERS[-1]) / (R_AIR *
   ↪  temperature)) / (R_AIR * temperature)
48         rho = ca.if_else(geopotential >= LAYERS[-1], upper_rho, rho)
49     return rho
50
51
52 def dynamic_pressure(p):
53     """Air-relative dynamic pressure of a scaled state, for optional path
   ↪ constraints."""
54     state = ca.SX.sym("y", 7)
55     physical = ca.DM(SCALE) * state
56     q = 0.5 * density(physical[1], p) * ((physical[2] - p["windMps"])**2 +
   ↪ physical[3]**2)
57     return ca.Function("dynamic_pressure", [state], [q])
58
```

```
59
60 def p1_rhs(p):
61     """Symbolic transcription of planar.mjs and atmosphere.mjs for derivatives."""
62     state = ca.SX.sym("y", 7)
63     u = ca.SX.sym("u", 2)
64     _, h, vx, vh, mass, theta, omega = ca.vertsplit(state)
65     rho = density(h, p)
66
67     length, diameter = p["lengthM"], p["diameterM"]
68     cos_theta, sin_theta = ca.cos(theta), ca.sin(theta)
69     axial = (vx - p["windMps"]) * cos_theta + vh * sin_theta
70     normal = -(vx - p["windMps"]) * sin_theta + vh * cos_theta
71     axial_force = -0.5 * rho * p["axialCd"] * np.pi * diameter**2 / 4 * axial * ca
   ↪ .fabs(axial)
72     strip_width = length / p["strips"]
73     arm = ca.DM(-length / 2 + (np.arange(p["strips"]) + 0.5) * strip_width + p["
   ↪ cpOffsetM"])
74     inertial_pitch_rate = omega - vx / (R + h) if p.get("geometryModel") == "
   ↪ spherical" else omega
75     local_velocity = normal + inertial_pitch_rate * arm
76     strip_force = -0.5 * rho * p["crossflowCd"] * diameter * strip_width *
   ↪ local_velocity * ca.fabs(local_velocity)
77     normal_force = ca.sum1(strip_force)
78     thrust = u[0] * p["maxThrustN"]
79     delta = u[1] * np.deg2rad(p["gimbalLimitDeg"])
80     dry_inertia = p["dryMassKg"] * (3 * (diameter / 2)**2 + length**2) / 12
81     force_x = (axial_force * cos_theta - normal_force * sin_theta + thrust * ca.
   ↪ cos(theta + delta)) / mass
82     force_h = (axial_force * sin_theta + normal_force * cos_theta + thrust * ca.
   ↪ sin(theta + delta)) / mass
83     angular_acceleration = (ca.dot(arm, strip_force) - length / 2 * thrust * ca.
   ↪ sin(delta)) / dry_inertia
84     horizontal_rate = vx
85     acceleration_x = force_x
86     acceleration_h = -G0 * (R / (R + h))**2 + force_h
87     if p.get("geometryModel") == "spherical":
88         radius = R + h
89         horizontal_rate = R / radius * vx
90         acceleration_x -= vx * vh / radius
91         acceleration_h += vx**2 / radius
92         angular_acceleration += force_x / radius - 2 * vx * vh / radius**2
93     return ca.Function("p1_scaled_rhs", [state, u], [ca.vertcat(
94         horizontal_rate, vh, acceleration_x, acceleration_h,
95         -thrust / (G0 * p["ispS"]), omega, angular_acceleration,
96     ) / ca.DM(SCALE)])
97
98 def seed_knots(seed, phase, fraction, gimbal_limit):
99     start = phase["startS"]
100    end = phase["endS"]
101    time = start + (end - start) * fraction
102    rows = [row for row in seed["trace"] if row["phase"] == phase["name"]
103            and start - 1e-8 <= row["t"] <= end + 1e-8]
104    source = np.array([row["t"] for row in rows])
105    states = np.array([row["state"] for row in rows])
106    controls = np.array([[row["throttle"], row["gimbalRad"] / gimbal_limit] for
   ↪ row in rows])
107    return (np.array([np.interp(time, source, v) for v in states.T]) / SCALE[:,
   ↪ None],
```

```
            np.array([np.interp(time, source, v) for v in controls.T]))


def rk_interval(rhs, substeps):
    """RK4 transcription of a piecewise-linear control interval."""
    state = ca.MX.sym("state", 7)
    u0 = ca.MX.sym("u0", 2)
    u1 = ca.MX.sym("u1", 2)
    duration = ca.MX.sym("duration")
    y = state
    h = duration / substeps
    for i in range(substeps):
        u = u0 + (u1 - u0) * (i / substeps)
        middle = u0 + (u1 - u0) * ((i + 0.5) / substeps)
        end = u0 + (u1 - u0) * ((i + 1) / substeps)
        k1 = rhs(ca.DM(SCALE) * y, u)
        k2 = rhs(ca.DM(SCALE) * (y + h * k1 / 2), middle)
        k3 = rhs(ca.DM(SCALE) * (y + h * k2 / 2), middle)
        k4 = rhs(ca.DM(SCALE) * (y + h * k3), end)
        y = y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
    return ca.Function("rk4_interval", [state, u0, u1, duration], [y])


def optimize(p, contract, seed, step, max_iter, *, shooting=False, altitude_buffer
    ↪ =0, substeps=8,
             fixed_initial_mass=False, fixed_ship=None, throttle_ceiling=1,
    ↪ gimbal_fraction=1,
             control_smoothing=0, touchdown_vertical_speed=-1.5, max_q=None,
    ↪ max_wall_time=140):
    if not p["minThrottle"] <= throttle_ceiling <= 1 or not 0 < gimbal_fraction <=
    ↪  1 or control_smoothing < 0:
        raise ValueError("Planning limits must fit inside the reference actuator
    ↪ limits")
    nlp = ca.Opti()
    rhs = p1_rhs(p)
    step_function = rk_interval(rhs, substeps) if shooting else None
    scale = ca.repmat(ca.DM(SCALE), 1, 1)
    gimbal_limit = np.deg2rad(p["gimbalLimitDeg"])
    floor = p["dryMassKg"] + contract["budget"]["unusableResidualKg"] + contract["
    ↪ touchdown"]["usableReserveMinKg"]
    top = 200_000 if p.get("upperAtmosphere") == "isothermal" else LAYERS[-1]
    altitude_limit = R * top / (R - top) - altitude_buffer
    stages = []
    q_function = dynamic_pressure(p) if max_q is not None else None

    def bounds(states, clearance=False):
        nlp.subject_to(nlp.bounded(p["deckHeightM"] / SCALE[1], states[1, :],
    ↪ altitude_limit / SCALE[1]))
        nlp.subject_to(states[4, :] >= floor / SCALE[4])
        if q_function is not None:
            nlp.subject_to(q_function.map(states.shape[1])(states) / 1e5 <= max_q
    ↪ / 1e5)
        if clearance:
            theta = states[5, :] * SCALE[5]
            extent = p["lengthM"] / 2 * ca.fabs(ca.sin(theta)) + p["diameterM"] /
    ↪ 2 * ca.fabs(ca.cos(theta))
            nlp.subject_to(states[1, :] >= (p["deckHeightM"] + extent) / SCALE[1])

```

```
157     for phase in seed["phases"]:
158         guess_duration = phase["endS"] - phase["startS"]
159         powered = phase["powered"]
160         intervals = max(5, int(np.ceil(guess_duration / (step if powered else step
    ↪  * 1.4))))
161         fraction = np.linspace(0, 1, intervals + 1)
162         if not powered:
163             fraction = 1 - (1 - fraction)**1.6
164         duration = nlp.variable()
165         lower = max(0.001, guess_duration * 0.85) if shooting else 0.001
166         upper = min(400, guess_duration * 1.15) if shooting else 400
167         nlp.subject_to(nlp.bounded(lower / 100, duration, upper / 100))
168         nlp.set_initial(duration, guess_duration / 100)
169         x = nlp.variable(7, intervals + 1)
170         x_guess, u_guess = seed_knots(seed, phase, fraction, gimbal_limit)
171         nlp.set_initial(x, x_guess)
172         if stages:
173             nlp.subject_to(x[:, 0] == stages[-1][2][:, -1])
174         else:
175             fixed = np.arange(7) if fixed_initial_mass else np.r_[0:4, 5:7]
176             nlp.subject_to(x[fixed, 0] == np.array(seed["initialState"])[fixed] /
    ↪ SCALE[fixed])
177         if powered:
178             u = nlp.variable(2, intervals + 1)
179             nlp.subject_to(nlp.bounded(p["minThrottle"], u[0, :], throttle_ceiling
    ↪ ))
180             nlp.subject_to(nlp.bounded(-gimbal_fraction, u[1, :], gimbal_fraction)
    ↪ )
181             nlp.set_initial(u, u_guess)
182         else:
183             u = ca.DM.zeros(2, intervals + 1)
184         physical_x = ca.repmat(scale, 1, intervals + 1) * x
185         f = rhs.map(intervals + 1)(physical_x, u)
186         durations = 100 * duration * ca.DM(np.diff(fraction)).T
187         dt = ca.repmat(durations, 7, 1)
188         mid = (x[:, :-1] + x[:, 1:]) / 2 + dt / 8 * (f[:, :-1] - f[:, 1:])
189         if shooting:
190             integrated = step_function.map(intervals)(x[:, :-1], u[:, :-1], u[:,
    ↪ 1:], durations)
191             nlp.subject_to(x[:, 1:] == integrated)
192             nlp.subject_to(nlp.bounded(-0.15, x[5, :] - ca.DM(x_guess[5:6, :]),
    ↪ 0.15))
193         else:
194             mid_f = rhs.map(intervals)(ca.repmat(scale, 1, intervals) * mid, (u[:,
    ↪  :-1] + u[:, 1:]) / 2)
195             nlp.subject_to(x[:, 1:] - x[:, :-1] == dt / 6 * (f[:, :-1] + 4 * mid_f
    ↪  + f[:, 1:]))
196         bounds(x, clearance=phase is not seed["phases"][-1])
197         if phase is seed["phases"][-1]:
198             bounds(x[:, :-1], clearance=True)
199         bounds(mid, clearance=True)
200         stages.append((phase, duration, x, u, fraction))
201 
202     first, final = stages[0][2][:, 0], stages[-1][2][:, -1]
203     # Interior targets leave room for numerical differences on replay.
204     nlp.subject_to(final[1:4] == np.array([p["deckHeightM"] + p["lengthM"] / 2, 0,
    ↪  touchdown_vertical_speed]) / SCALE[1:4])
205     upright = 0.5 + 2 * round((seed["finalState"][5] - np.pi / 2) / (2 * np.pi))
```

```
    nlp.subject_to(final[5:7] == [upright, 0])
    nlp.subject_to(final[4] >= (floor + 75) / SCALE[4])
    if fixed_ship is not None:
        nlp.subject_to(final[0] == fixed_ship / SCALE[0])
    objective = first[4] - final[4] if fixed_initial_mass else first[4]
    if control_smoothing:
        objective += control_smoothing * sum(ca.sumsqr(stage[3][:, 1:] - stage[3][:, :-1]) for stage in stages if stage[0]["powered"])
    nlp.minimize(objective)
    nlp.solver("ipopt", {"print_time": False}, {"print_level": 0, "max_iter": max_iter,
                "max_wall_time": max_wall_time, "tol": 1e-8, "acceptable_tol": 2e-6,
                "acceptable_iter": 10, "bound_relax_factor": 0})

    solution = nlp.solve_limited()
    info = solution.stats()
    phases = []
    elapsed = 0.0
    for phase, duration, x, u, fraction in stages:
        time = 100 * float(solution.value(duration))
        controls = np.asarray(solution.value(u))
        states = (np.asarray(solution.value(x)) * SCALE[:, None]).T
        phases.append(dict(name=phase["name"], powered=phase["powered"], knots=(elapsed + time * fraction).tolist(),
                           states=states.tolist(), throttle=controls[0].tolist(),
                           gimbalRad=(controls[1] * gimbal_limit).tolist()))
        elapsed += time
    return dict(status=info["return_status"], success=bool(info["success"]), iterations=info["iter_count"],
                method="RK4 multiple shooting" if shooting else "Hermite-Simpson collocation",
                atmosphereBufferM=altitude_buffer, rkSubsteps=substeps if shooting else None,
                maxIntervalTargetS=step, propellantKg=float(solution.value(first[4]) * SCALE[4] - p["dryMassKg"]),
                shipLocationM=phases[-1]["states"][-1][0], phases=phases)
```

Listing 4: Control interpolation and touchdown replay

```
export function scheduledPhases(schedule) {
  const phases = schedule.map(phase => {
    const times = phase.knots;
    function command(t) {
      if (!phase.powered) return off();
      let left = 0;
      let right = times.length - 1;
      while (right - left > 1) {
        const mid = (left + right) >> 1;
        if (t >= times[mid]) left = mid;
        else right = mid;
      }
      const fraction = clamp((t - times[left]) / (times[right] - times[left]), 0, 1);
      return {throttle: phase.throttle[left] + fraction * (phase.throttle[right] - phase.throttle[left]),
        gimbalRad: phase.gimbalRad[left] + fraction * (phase.gimbalRad[right] - phase.gimbalRad[left])};
    }
```

```
73      return {name: phase.name, powered: phase.powered, command, duration: times.at
    ↪ (-1) - times[0]};
74    });
75    // Preserve the P2 allowance for contact just after the last control knot.
76    if (phases.at(-1).powered) phases.at(-1).duration += 0.5;
77    return phases;
78  }
79
80  export function replay(p, contract, propellant, phases, {step = 0.02, samples =
    ↪ false, ship = null, startState = null, onRecord = null} = {}) {
81    let y = startState === null ? initialState(p, contract, propellant) : [...
    ↪ startState];
82    const initial = [...y];
83    let time = 0;
84    let event = 'schedule_ended';
85    let peakQ = {value: 0};
86    let peakLoad = {value: 0};
87    let maxAltitudeM = y[1];
88    let minimumClearanceM = clearance(y, p);
89    let maxMach = 0;
90    let ignitionCount = 0;
91    let powered = false;
92    const trace = [];
93    const executed = [];
94    const deckVelocity = contract.environment.deckVelocityMps;
95
96    function record(phase, force = false) {
97      const a = aero(y, p);
98      const u = phase.command(time, y, a);
99      const thrust = u.throttle * p.maxThrustN;
100     const angle = y[5] + u.gimbalRad;
101     const load = Math.hypot(a.fx + thrust * Math.cos(angle), a.fz + thrust * Math.
    ↪ sin(angle)) / (y[4] * g0);
102     if (onRecord) onRecord({timeS: time, state: y, phase: phase.name, powered:
    ↪ phase.powered, aero: a, control: u, loadG: load});
103     if (a.q > peakQ.value) peakQ = {value: a.q, timeS: time, altitudeM: y[1]};
104     if (load > peakLoad.value) peakLoad = {value: load, timeS: time, altitudeM: y
    ↪ [1]};
105     maxMach = Math.max(maxMach, a.mach);
106     maxAltitudeM = Math.max(maxAltitudeM, y[1]);
107     minimumClearanceM = Math.min(minimumClearanceM, clearance(y, p));
108     if (samples && (force || !trace.length || time - trace.at(-1).t >= 0.18)) {
109       trace.push({t: time, phase: phase.name, state: [...y], throttle: u.throttle,
110         gimbalRad: u.gimbalRad, qPa: a.q, mach: a.mach, loadG: load});
111     }
112   }
113
114   try {
115     flight: for (const specifiedPhase of phases) {
116       const period = specifiedPhase.samplePeriodS;
117       if (period !== undefined && (!Number.isFinite(period) || period <= 0)) {
118         throw new Error('A sampled controller needs a positive finite period');
119       }
120       let heldCommand;
121       let nextSample = time;
122       const phase = period === undefined ? specifiedPhase : {
123         ...specifiedPhase, command: () => heldCommand,
124       };
```

```
    const updateControl = () => {
      if (period !== undefined && time >= nextSample - 1e-10) {
        heldCommand = specifiedPhase.command(time, y, aero(y, p));
        nextSample += period;
      }
    };
    const start = time;
    const startState = [...y];
    const finish = start + (phase.duration ?? (600 - start));
    if (phase.altitude !== undefined && y[1] < phase.altitude && y[3] < 0) {
      event = 'missed_ignition_altitude';
      break;
    }
    if (phase.powered && !powered) ignitionCount++;
    powered = phase.powered;
    updateControl();
    record(phase, true);
    let phaseEnded = false;
    while (time < finish - 1e-9) {
      updateControl();
      const dt = Math.min(step, finish - time, period === undefined ? Infinity :
↪  nextSample - time);
      const next = rk4(time, y, dt, p, phase.command);
      if (!next.every(Number.isFinite)) {
        event = 'nonfinite_state';
        break flight;
      }
      const ignition = phase.altitude !== undefined && y[1] >= phase.altitude &&
↪  next[1] <= phase.altitude;
      const contact = clearance(next, p) <= 0;
      const depleted = next[4] <= p.dryMassKg;
      if (ignition || contact || depleted) {
        const surface = ignition ? state => state[1] - phase.altitude
          : contact ? state => clearance(state, p) : state => state[4] - p.
↪ dryMassKg;
        let lo = 0;
        let hi = dt;
        for (let i = 0; i < 24; i++) {
          const mid = (lo + hi) / 2;
          if (surface(rk4(time, y, mid, p, phase.command)) > 0) lo = mid;
          else hi = mid;
        }
        y = rk4(time, y, hi, p, phase.command);
        time += hi;
        phaseEnded = true;
        if (!ignition) event = contact ? 'contact' : 'fuel_exhausted';
        break;
      }
      y = next;
      time += dt;
      record(phase);
    }
    record(phase, true);
    executed.push({name: phase.name, powered, startS: start, endS: time,
↪ startState,
      endState: [...y], consumedKg: startState[4] - y[4]});
    if (event === 'contact' || event === 'fuel_exhausted') break;
    if (phase.altitude !== undefined && !phaseEnded) break;
```

```
    }
  } catch (error) {
    if (error.message !== 'Atmosphere altitude outside implemented range') throw error;
    event = 'atmosphere_out_of_range';
  }

  const touchdown = contract.touchdown;
  const target = ship ?? y[0];
  const remainder = y[4] - p.dryMassKg;
  const residual = contract.budget.unusableResidualKg;
  const values = {shipErrorM: Math.abs(y[0] - target), horizontalMps: Math.abs(y[2] - deckVelocity[0]),
    verticalMps: Math.abs(y[3] - deckVelocity[1]), tiltDeg: Math.abs(wrap(y[5] - Math.PI / 2)) / rad,
    pitchRateDegS: Math.abs(y[6]) / rad, reserveKg: remainder - residual};
  const violations = {
    contact: event !== 'contact',
    ship: values.shipErrorM > touchdown.shipTargetToleranceM,
    horizontal: values.horizontalMps > touchdown.horizontalSpeedMaxMps,
    vertical: values.verticalMps > touchdown.verticalSpeedMaxMps,
    tilt: values.tiltDeg > touchdown.tiltMaxDeg,
    pitchRate: values.pitchRateDegS > touchdown.pitchRateMaxDegS,
    reserve: values.reserveKg < touchdown.usableReserveMinKg,
    ignitions: ignitionCount > contract.operations.maximumIgnitions,
  };
  const methane = propellant / (1 + p.oxygenFuelRatio);
  return {event, passes: !Object.values(violations).some(Boolean), violations, propellantKg: propellant,
    methaneKg: methane, liquidOxygenKg: propellant - methane, consumedKg: initial[4] - y[4],
    reserveKg: touchdown.usableReserveMinKg, residualKg: residual,
    extraRemainingKg: remainder - residual - touchdown.usableReserveMinKg, timeS: time,
    initialState: initial, finalState: y, shipLocationM: target, touchdown: values,
    clearanceM: clearance(y, p), minimumClearanceM, ignitionCount, phases: executed,
    peakQPa: peakQ, peakLoadG: peakLoad, maxAltitudeM, maxMach, stepS: step, trace};
}
```

Listing 5: Spherical trajectory replay entry point

```
import fs from 'node:fs';
import {readJSON,replay,scheduledPhases} from './p2-flight.mjs';
const req=JSON.parse(fs.readFileSync(0,'utf8'));
const p={...readJSON('config/spherical.json'),...req.change};
const result=replay(p,readJSON('config/p0-contract.json'),req.candidate.propellantKg,scheduledPhases(req.candidate.phases),{step:req.stepS??.005,ship:req.candidate.shipLocationM,samples:req.samples??false});
console.log(JSON.stringify(result));
```

Report on the Use of AI

OpenAI Codex (`gpt-6.1-sol`)

Query: "Plan adjustments and corrections based on the cause, then act." **Output:** Implemented replanning at separation; ten cases passed, and an angle-wrapping error in the initial guess for one case was found and fixed.

Query: "Do it the way you said." **Output:** Built a reference with 95% thrust and 9° gimbal planning limits, ignition 1 s earlier, and 20 ms state feedback; five unknown wind errors from -0.1 to $+0.1$ m/s landed within about 2.95 m with 6.03–6.04 t remaining of 30 t, assuming exact state knowledge and instantaneous actuators.

Kiro in DeepSeek Harness (`claude-opus-5-5`)

Query: "Organize the problems and gaps we still have into a task document, then revise item by item according to that task document yourself." **Output (code only):** Added an optional isothermal upper atmosphere and an optional dynamic-pressure path constraint.